

Explicacion de simulacion.py

TP Simulacion de Colas — Grupo 22
Sistema de Registro Dactilar — Municipalidad de Rio Cuarto

1. Vision general de simulacion.py

simulacion.py es el nucleo logico de la aplicacion. Implementa una **simulacion de eventos discretos (DES)** pura: no tiene interfaz grafica, no habla con bases de datos ni con la red. Solo recibe parametros, simula el sistema y devuelve una lista de filas con el estado del sistema en cada evento.

El sistema simulado es la sala de registro dactilar de la Municipalidad de Rio Cuarto: empleados que llegan, hacen fila, son atendidos en terminales, y un tecnico que periodicamente mantiene esas terminales.

El archivo se organiza en siete bloques:

1. Constantes y variables globales.
2. Generadores de variables aleatorias.
3. Cola de eventos (heap binario).
4. Funciones auxiliares de consulta del estado.
5. Funciones de asignacion y resolucion de prioridad.
6. Procesadores de eventos (uno por tipo de evento).
7. Loop principal y reset.

La separacion entre **simulacion.py** y **main.py** es intencional: la logica de simulacion no sabe nada de la GUI, y la GUI no sabe como funciona la simulacion. Se comunican solo a traves de **vector_estado**.

2. Constantes y variables globales

2.1 Constantes de configuracion

```
MAX_COLA          = 5          # maximo de empleados en cola de espera
MAX_ITER          = 99999      # maximo de eventos a procesar
MEDIA_LLEGADA_EMP = 2.0        # media de la exponencial de llegada de empleados
ATN_MIN, ATN_MAX  = 5, 8       # rango de tiempo de atencion (uniforme)
MANT_MIN, MANT_MAX = 3, 10     # rango de tiempo de mantenimiento (uniforme)
TEC_MIN, TEC_MAX  = 57, 63     # rango de tiempo entre llegadas del tecnico (uniforme)
```

Estas constantes son los parametros del sistema. **main.py** las sobrescribe antes de cada simulacion con los valores que el usuario ingreso en la interfaz. Si el usuario no cambia nada, corren con estos valores por defecto.

2.2 Estructura: terminales

Lista de 4 diccionarios, uno por terminal. Cada terminal tiene:

```
{"id": 1, "estado": "Libre", "pendiente": True, "fin_aten": None, "emp_id": None}
```

- **id**: numero de terminal (1 a 4).
- **estado**: puede ser *Libre*, *Ocupada* o *Siendo mantenida*.
- **pendiente**: flag booleano que indica si esa terminal todavia necesita mantenimiento en esta ronda del tecnico. Arranca en True para todas.
- **fin_aten**: el tiempo de reloj en que termina la atencion actual (None si libre).
- **emp_id**: ID del empleado que esta siendo atendido (None si libre).

2.3 Estructura: tecnico

```
tecnico = {"estado": "Descansando", "terminal_id": None, "fin_manten": None}
```

Diccionario unico que representa al tecnico. Sus estados posibles son:

- **Descansando**: termino la ronda de mantenimiento y espera su proximo turno.
- **Esperando Terminal Libre**: llego para mantener pero todas las terminales que le faltan estan ocupadas. Espera al costado sin entrar a la cola de empleados.
- **Realizando Mantenimiento**: esta trabajando sobre una terminal.

2.4 Estructura: cola

Lista de diccionarios que representa la fila de espera de empleados. Cada elemento tiene solo dos campos: **tipo** (siempre "empleado") e **id**. El tecnico **nunca** entra a esta cola; tiene su propio mecanismo de espera.

```
cola = [{"tipo": "empleado", "id": 3}, {"tipo": "empleado", "id": 5}, ...]
```

2.5 Estructura: empleados

Diccionario global con un registro por cada empleado que esta actualmente en el sistema (esperando o siendo atendido). La clave es el emp_id:

```
empleados = {  
    3: {"id": 3, "col": 3, "estado": "EA",  
        "hora_inicio_espera": 4.72, "terminal_id": None},  
    5: {"id": 5, "col": 5, "estado": "SA",  
        "hora_inicio_espera": None, "terminal_id": 2}  
}
```

- **col**: numero de columna que ocupa en la tabla (igual al ID de llegada).
- **estado**: EA (esperando atencion) o SA (siendo atendido).
- **hora_inicio_espera**: el reloj en que entro a la cola. None si fue atendido de inmediato o si ya salio de la cola.
- **terminal_id**: la terminal donde esta siendo atendido. None si todavia espera.

Los empleados rechazados (RT) nunca entran a este diccionario. Los que terminan de ser atendidos se borran despues de guardar la fila. Por eso los huecos en la numeracion de columnas corresponden a rechazados.

2.6 Cola de eventos: eventos

Lista que funciona como un **heap binario minimo** (administrada con el modulo *heapq*). Cada elemento es una tupla de 4 valores:

```
(tiempo, seq, tipo, eid)
```

- **tiempo**: el momento del reloj en que ocurre el evento.
- **seq**: numero secuencial monotono para desempatar eventos con el mismo tiempo (se respeta el orden de insercion, igual que el antiguo min() lineal).
- **tipo**: string que identifica el tipo: "llegada_emp", "fin_atencion", "llegada_tec" o "fin_manten".
- **eid**: dato extra, generalmente el ID de la terminal involucrada (None para llegadas).

2.7 Contadores y acumuladores estadisticos

```
reloj          = 0.0    # tiempo actual de la simulacion
iteracion      = 0      # cantidad de eventos procesados
id_emp_contador = 0      # ultimo ID de empleado asignado
contador_atendidos = 0   # empleados que completaron su atencion
contador_rt     = 0      # empleados rechazados (cola llena)
contador_llegaron = 0    # total de empleados que llegaron al sistema
contador_espera  = 0      # empleados incluidos en el promedio de espera
acum_espera     = 0.0    # suma de tiempos de espera de todos los atendidos
```

Con estos valores se calculan las dos metricas pedidas por el enunciado:

- **% que se van** = $\text{contador_rt} / \text{contador_llegaron} * 100$
- **Promedio de espera** = $\text{acum_espera} / \text{contador_espera}$

contador_espera incluye a los empleados que esperaron 0 minutos (fueron atendidos de inmediato). Esto hace que el promedio sea sobre todos los empleados del sistema, no solo los que esperaron.

2.8 Otras variables globales

```
ultimo_idx_terminal = -1 # puntero round-robin para asignar terminales
_seq                = 0   # contador monotono para desempate en el heap
vector_estado       = []  # lista de filas guardadas, resultado final
```

3. Generadores de variables aleatorias

Todos los tiempos aleatorios del sistema se generan con estas cuatro funciones. Cada una devuelve una tupla (**rnd, tiempo**): el numero aleatorio usado y el tiempo calculado. Devolver ambos permite mostrar el RND exacto en la tabla.

3.1 trunc2 — truncado a 2 decimales

```
def trunc2(x):
    return int(x * 100) / 100
```

Trunca un numero a 2 decimales sin redondear. Se aplica al RND antes de la formula y al resultado, para que el numero mostrado en la tabla sea exactamente el que se uso en el calculo. Sin esto, la tabla mostraria 0.26 pero la formula habria corrido con 0.2634...

El enunciado pide truncar, no redondear. trunc2 usa int() que siempre descarta los decimales restantes sin importar el valor.

3.2 gen_llegada_emp — distribucion exponencial negativa

```
def gen_llegada_emp():
    rnd = trunc2(random.random())
    t = trunc2(-MEDIA_LLEGADA_EMP * math.log(1 - rnd))
    return rnd, t
```

Genera el tiempo hasta la proxima llegada de un empleado. Usa la formula de la inversa de la exponencial negativa: **$t = -\text{media} * \ln(1 - \text{rnd})$** . Con media = 2 minutos, los tiempos entre llegadas son pequenos pero variables.

3.3 gen_atencion y gen_mantenimiento — distribucion uniforme

```
def gen_atencion():
    rnd = trunc2(random.random())
    t = trunc2(ATN_MIN + rnd * (ATN_MAX - ATN_MIN))
    return rnd, t
```

Formula de la uniforme: **$t = \text{min} + \text{rnd} * (\text{max} - \text{min})$** . Para atencion: entre 5 y 8 minutos. Para mantenimiento: entre 3 y 10 minutos. El tecnico genera un nuevo RND por cada terminal que mantiene en la ronda.

3.4 gen_llegada_tec — uniforme para el tecnico

Igual que las anteriores pero con rango 57-63 minutos. Se llama para programar la proxima llegada del tecnico una vez que termina su ronda completa. El tecnico llega aproximadamente cada hora.

4. Cola de eventos — el heap binario

El corazon de cualquier simulacion de eventos discretos es la estructura que decide cual es el proximo evento a procesar. Aqui se usa un **heap binario minimo** a traves del modulo *heapq* de Python.

Un heap minimo garantiza que el elemento con menor valor siempre esta en la cima. Como los eventos se ordenan por tiempo, el proximo evento a procesar siempre es el primero de la estructura. Insertar o extraer un elemento cuesta **$O(\log n)$** , mucho mas eficiente que buscar el minimo en una lista no ordenada ($O(n)$).

4.1 push_evento — insertar un evento

```
def push_evento(tiempo, tipo, eid=None):
    global _seq
    heapq.heappush(eventos, (tiempo, _seq, tipo, eid))
    _seq += 1
```

Cada evento se inserta como una tupla (tiempo, seq, tipo, eid). El campo **seq** es un contador que crece con cada insercion. Cuando dos eventos tienen el mismo tiempo, Python los compara por el segundo campo de la tupla (seq), y el que fue insertado primero tiene menor seq y sale primero.

Esto replica exactamente el comportamiento del antiguo `min()` lineal sobre una lista en orden de insercion.

4.2 siguiente_evento — extraer el proximo

```
def siguiente_evento():
    return heapq.heappop(eventos)
```

Extrae y devuelve el evento con menor tiempo. Internamente el heap se reestructura en $O(\log n)$ para mantener su propiedad.

4.3 tiempo_de — consultar sin extraer

```
def tiempo_de(tipo):
    tiempos = [e[0] for e in eventos if e[2] == tipo]
    return min(tiempos) if tiempos else None
```

No extrae ningun evento. Busca en el heap el minimo tiempo de los eventos de un tipo especifico. Se usa para mostrar en la tabla los campos 'Prox. Evento' de llegada de empleado y de llegada de tecnico.

5. Funciones auxiliares de consulta del estado

Estas funciones no modifican el estado; solo lo consultan y devuelven una respuesta. Son la capa de abstraccion entre el estado global y los procesadores de eventos.

5.1 terminal_libre_para_emp — round-robin

```
def terminal_libre_para_emp():
    global ultimo_idx_terminal
    n = len(terminales)
    for offset in range(1, n + 1):
        idx = (ultimo_idx_terminal + offset) % n
        if terminales[idx]['estado'] == 'Libre':
            ultimo_idx_terminal = idx
            return terminales[idx]
    return None
```

Busca una terminal libre usando **round-robin**: en lugar de siempre empezar desde la terminal 1, arranca desde la siguiente a la que fue asignada la ultima vez. Esto distribuye la carga entre terminales de forma equitativa.

El modulo % hace que el indice sea circular: si la ultima asignada fue la terminal 4 (indice 3), la siguiente busqueda empieza por la terminal 1 (indice 0). Si no hay ninguna libre, devuelve None.

El round-robin es deterministico: no usa numeros aleatorios. La distribucion es predecible y reproducible.

5.2 terminal_libre_con_pendiente — para el tecnico

```
def terminal_libre_con_pendiente():
    for t in terminales:
        if t['estado'] == 'Libre' and t['pendiente']:
```

```
        return t
    return None
```

Busca una terminal que este libre Y que todavia tenga mantenimiento pendiente en esta ronda. El tecnico solo va a terminales que cumplen ambas condiciones. No usa round-robin: toma la primera que encuentra.

5.3 hay_pendiente_ocupada

```
def hay_pendiente_ocupada():
    return any(t['pendiente'] and t['estado'] == 'Ocupada' for t in terminales)
```

Devuelve True si hay al menos una terminal que esta ocupada (con empleado) Y todavia tiene mantenimiento pendiente. Se usa para saber si el tecnico debe quedarse esperando o irse a descansar.

5.4 emp_enCola

```
def emp_enCola():
    return len(cola)
```

Devuelve cuantos empleados hay esperando en la cola. Se compara contra MAX_COLA para decidir si un empleado nuevo puede entrar o si debe ser rechazado.

6. Funciones de asignacion y prioridad

6.1 asignar_empleado_a_terminal

```
def asignar_empleado_a_terminal(emp_id, terminal):
    rnd_a, t_a = gen_atencion()
    fin = round(reloj + t_a, 2)

    terminal['estado'] = 'Ocupada'
    terminal['fin_aten'] = fin
    terminal['emp_id'] = emp_id

    empleados[emp_id]['estado'] = 'SA'
    empleados[emp_id]['terminal_id'] = terminal['id']

    push_evento(fin, 'fin_atencion', terminal['id'])
    return rnd_a, t_a
```

Hace todo lo necesario para que un empleado empiece a ser atendido: genera el tiempo de atencion, marca la terminal como Ocupada, actualiza el estado del empleado a SA (Siendo Atendido) y programa el evento fin_atencion en el heap para que la simulacion sepa cuando va a terminar.

6.2 asignar_tecnico_a_terminal

```
def asignar_tecnico_a_terminal(terminal):
    rnd_m, t_m = gen_mantenimiento()
    fin = round(reloj + t_m, 2)
```

```

terminal['estado']      = 'Siendo mantenida'
terminal['pendiente']    = False
tecnico['estado']        = 'Realizando Mantenimiento'
tecnico['terminal_id']   = terminal['id']
tecnico['fin_manten']    = fin

push_evento(fin, 'fin_manten', terminal['id'])
return rnd_m, t_m

```

Analogo al anterior pero para el tecnico. El flag **pendiente** se pone en False en este momento porque el mantenimiento de esa terminal ya esta en curso: no hay que volver a hacerlo en esta ronda.

6.3 atenderColaConTerminal — la regla de prioridad

Esta es la funcion mas importante del archivo en terminos de logica de negocio. Se llama cada vez que una terminal queda disponible (ya sea porque termino una atencion o porque termino un mantenimiento) y decide que hacer con ella.

```

def atenderColaConTerminal(terminal):
    # Prioridad 1: tecnico esperando y terminal pendiente
    if tecnico['estado'] == 'Esperando Terminal Libre' and terminal['pendiente']:
        rnd_m, t_m = asignarTecnicoATerminal(terminal)
        return {'manten': rnd_m, 't_manten': t_m}

    # Prioridad 2: primer empleado en cola
    if cola:
        emp = cola.pop(0)
        espera = round(reloj - empleados[emp['id']]['hora_inicio_espera'], 2)
        acum_espera += espera
        contador_espera += 1
        rnd_a, t_a = asignarEmpleadoATerminal(emp['id'], terminal)
        return {'atencion': rnd_a, 't_atencion': t_a}

    # Nadie espera
    terminal['estado'] = 'Libre'
    return {}

```

Las tres posibilidades en orden de prioridad:

Prioridad 1 — el tecnico tiene preferencia. Si el tecnico esta en estado 'Esperando Terminal Libre' Y la terminal que se libero tiene mantenimiento pendiente, el tecnico la toma de inmediato antes que cualquier empleado en la cola. El tecnico no interrumpe atenciones en curso, pero en cuanto una terminal se libera y le corresponde, la toma primero.

Prioridad 2 — empleados en cola. Si no hay tecnico esperando (o la terminal no le corresponde), se toma el primer empleado de la cola. En ese momento exacto se calcula su tiempo de espera (reloj actual menos hora_inicio_espera) y se acumula. El empleado pasa de EA a SA.

Sin prioridad — terminal libre. Si no hay nadie esperando, la terminal simplemente queda en estado Libre.

La espera se calcula y acumula cuando el empleado SALE de la cola y empieza a ser atendido, no cuando llega ni cuando termina. Esto garantiza que el calculo sea exactamente la diferencia entre el momento de entrar a la cola y el momento de salir de ella.

7. snapshot_empleados y guardar_fila

7.1 snapshot_empleados

```
def snapshot_empleados():
    return {
        emp['col']: (emp['estado'], emp['hora_inicio_espera'], emp['terminal_id'])
        for emp in empleados.values()
    }
```

Toma una foto del estado de todos los empleados activos en el sistema en este momento. Devuelve un diccionario donde la clave es el numero de columna del empleado (= su ID de llegada) y el valor es una tupla de 3 elementos: estado, hora de inicio de espera y terminal asignada.

Este snapshot se guarda dentro de cada fila del vector de estado. Cuando la GUI quiere mostrar el estado de un empleado en una fila determinada, consulta este dict con el ID del empleado como clave.

7.2 guardar_fila

```
def guardar_fila(evento_nombre, rnds):
    vector_estado.append({
        'num': len(vector_estado),
        'reloj': reloj,
        'evento': evento_nombre,
        'prox_emp': tiempo_de('llegada_emp'),
        'prox_tec': tiempo_de('llegada_tec'),
        'term_est': [t['estado'] for t in terminales],
        'term_pend': [t['pendiente'] for t in terminales],
        'term_fin': [t['fin_aten'] for t in terminales],
        'fin_mant': tecnico['fin_manten'],
        'tec_est': tecnico['estado'],
        'tec_term': tecnico['terminal_id'],
        'cola': len(cola),
        'n_at': contador_atendidos,
        'n_lleg': contador_llegaron,
        'n_rt': contador_rt,
        'n_esp': contador_espera,
        'acum_esp': acum_espera,
        'rnd_le': rnds.get('llegada_emp'), ...
        'emp': snapshot_empleados(),
    })
```

Guarda el estado completo del sistema en el momento exacto de llamarla. Todos los valores son **crudos**: floats sin truncar, booleanos como True/False, None donde no hay valor. El formateo (truncado a 2 decimales, SI/NO, guiones, porcentajes, promedios) lo hace la GUI al momento de mostrar cada celda.

El parametro **rnds** es el diccionario con los numeros aleatorios generados en ese evento especifico. No todos los eventos generan los mismos RNDs: una llegada de empleado genera RND de llegada y puede generar RND de atencion; una llegada de tecnico puede generar RND de mantenimiento. `guardar_fila` guarda lo que le llega, y los campos que no corresponden quedan en `None`.

El numero de fila (`num`) es simplemente la longitud actual de `vector_estado` antes de agregar esta fila. La fila 0 es siempre la de Inicializacion.

8. Procesadores de eventos

Hay un procesador por cada tipo de evento posible en la simulacion. Cada uno recibe el control cuando el loop principal extrae su evento del heap, modifica el estado global segun las reglas del sistema, programa los eventos futuros que correspondan, y llama a `guardar_fila` para registrar el estado.

8.1 procesar_llegada_emp — llegada de un empleado

Es el procesador mas complejo porque tiene tres caminos posibles:

Camino 1 — hay terminal libre: el empleado es registrado en el dict con `hora_inicio_espera = None` (no espero), se lo asigna de inmediato a la terminal via round-robin, y se suma 1 a `contador_espera` con `espera = 0`. El promedio de espera lo incluye como si hubiera esperado 0 minutos.

Camino 2 — no hay terminal libre pero la cola tiene lugar: el empleado entra al dict con `hora_inicio_espera = reloj` (se anota el momento exacto) y se agrega al final de la lista cola. No se genera ningun evento nuevo para el: la cola se procesa cuando se libera una terminal.

Camino 3 — la cola esta llena ($\geq \text{MAX_COLA}$): el empleado es rechazado. Solo se incrementa `contador_rt`. No entra al dict, no ocupa columna en la tabla, no vuelve.

En los tres caminos, al final siempre se genera la proxima llegada de empleado y se la programa en el heap. La simulacion debe saber cuando llega el siguiente para poder procesarlo en el momento correcto.

8.2 procesar_fin_atencion — fin de atencion en una terminal

Se ejecuta cuando una terminal termina de atender a un empleado. El orden de operaciones es importante:

1. Se obtiene que terminal termino y que empleado estaba ahi.
2. Se incrementa `contador_atendidos`.
3. Se cambia el estado del empleado a "AT" (quedo visible en tabla).
4. Se limpia la terminal (`emp_id = None`, `fin_aten = None`).
5. Se llama a `atenderColaConTerminal`: decide si el tecnico, un empleado de la cola, o nadie toma esa terminal.
6. Se guarda la fila (el empleado todavia esta en el dict con estado AT).
7. Se borra el empleado del dict (desaparece de los snapshots futuros).

El empleado se borra despues de guardar la fila, no antes. Esto es lo que hace que la x roja aparezca en la fila de fin de atencion y desaparezca en las filas siguientes.

8.3 procesar_llegada_tec — llegada del tecnico

El tecnico llega periodicamente para hacer mantenimiento. Al llegar hay tres situaciones posibles:

Caso 1 — hay terminal libre con pendiente: el tecnico va directo a mantenerla. Se genera el tiempo de mantenimiento y se programa fin_manten en el heap.

Caso 2 — todas las terminales pendientes estan ocupadas: el tecnico no puede interrumpir. Se queda en estado 'Esperando Terminal Libre'. No entra a la cola de empleados. Cuando cualquier terminal ocupada-pendiente se libere, atenderColaConTerminal le dara prioridad sobre los empleados en espera.

Caso 3 — no hay ninguna terminal con pendiente = True: esto significa que ya no quedan terminales por mantener en esta ronda (todas tienen pendiente = False). El tecnico se va y se programa su proxima llegada en el heap.

8.4 procesar_fin_manten — fin de mantenimiento de una terminal

Cuando el tecnico termina de mantener una terminal, busca que hacer a continuacion. El proceso en orden:

1. Se limpia el estado del tecnico (terminal_id = None, fin_manten = None).
2. Se busca si hay otra terminal libre con pendiente (siguiente en la ronda).
3. Se busca si hay alguna terminal ocupada con pendiente (tecnico debe esperar).

Caso A — hay otra terminal libre con pendiente: la terminal que termino de ser mantenida queda Libre, se llama a atenderColaConTerminal para ver si hay empleados esperando, y el tecnico va directo a la siguiente terminal pendiente.

Caso B — hay terminales pendientes pero todas estan ocupadas: la terminal que termino queda Libre (y se atiende la cola si hay alguien), el tecnico queda en 'Esperando Terminal Libre' hasta que alguna se libere.

Caso C — no quedan terminales pendientes (ronda completa): el tecnico termino de mantener todas las terminales. Se resetean todos los flags pendiente a True (para la proxima ronda), el tecnico pasa a 'Descansando' y se programa su proxima llegada en el heap con gen_llegada_tec. La terminal que termino queda Libre.

El ciclo del tecnico es: llega -> mantiene terminal 1 -> mantiene terminal 2 -> ... -> mantiene terminal 4 -> descansa ~60 min -> llega de nuevo. Si una terminal esta ocupada cuando le toca, espera al costado sin bloquear la cola de empleados.

9. Loop principal — simular(tiempo_max)

Es el punto de entrada de la simulacion. Recibe el tiempo maximo en minutos y corre el loop de eventos hasta que se cumpla alguna condicion de corte.

9.1 Inicializacion

```

rnd_e, t_e = gen_llegada_emp()
push_evento(t_e, 'llegada_emp')

rnd_t, t_t = gen_llegada_tec()
push_evento(t_t, 'llegada_tec')

guardar_fila('Inicializacion', {...})

```

Antes de empezar el loop, se programan los dos primeros eventos: la primera llegada de empleado y la primera llegada del tecnico. Ambos se insertan en el heap con sus tiempos generados aleatoriamente. Luego se guarda la fila de inicializacion (fila 0) con el estado inicial.

9.2 El loop

```

while iteracion < MAX_ITER and reloj < tiempo_max:
    if not eventos:
        break

    tiempo, _, tipo, eid = siguiente_evento()

    if tiempo > tiempo_max:
        break

    reloj = tiempo

    if tipo == 'llegada_emp': procesar_llegada_emp()
    elif tipo == 'fin_atencion': procesar_fin_atencion(eid)
    elif tipo == 'llegada_tec': procesar_llegada_tec()
    elif tipo == 'fin_manten': procesar_fin_manten(eid)

    iteracion += 1

```

En cada iteracion el loop hace exactamente esto:

1. Extrae el evento con menor tiempo del heap (`siguiente_evento`).
2. Si ese tiempo supera `tiempo_max`, corta el loop SIN pisar el reloj (el reloj conserva el tiempo del ultimo evento efectivamente procesado).
3. Avanza el reloj al tiempo del evento.
4. Llama al procesador correspondiente segun el tipo de evento.
5. Incrementa el contador de iteraciones.

9.3 Condiciones de corte

El loop se detiene cuando ocurre cualquiera de estas condiciones:

- **iteracion** >= **MAX_ITER** (99.999 eventos procesados + fila 0 = 100.000 filas totales).
- **reloj** >= **tiempo_max** (al inicio del loop, antes de extraer).
- **El heap esta vacio** (no quedan eventos programados, situacion inusual).
- **El proximo evento supera tiempo_max** (al extraerlo, antes de procesar).

El limite de 100.000 filas existe para que la aplicacion no se cuelgue en simulaciones muy largas. En condiciones normales el corte ocurre por `tiempo_max`, no por `MAX_ITER`.

10. resetear_estado — preparar para una nueva simulacion

Antes de cada simulacion, **main.py** llama a **resetear_estado()** para limpiar todas las variables globales y partir desde cero. Sin esto, la segunda simulacion arrancaria con el estado final de la primera.

```
terminales = [
    {'id': i, 'estado': 'Libre', 'pendiente': True,
     'fin_aten': None, 'emp_id': None}
    for i in range(1, 5)
]
tecnico = {'estado': 'Descansando', 'terminal_id': None, 'fin_manten': None}
cola = []
empleados = {}
eventos = []
reloj = 0.0
# ... todos los contadores en 0
```

Recrea las 4 terminales desde cero con pendiente=True, reinicia el tecnico a 'Descansando', vacia la cola, el dict de empleados, el heap de eventos, y pone todos los contadores y acumuladores en cero.

11. Como se acumulan las estadisticas fila a fila

Las estadisticas no se calculan al final: se van acumulando evento a evento. Cada fila del vector de estado guarda una foto de los contadores en ese momento, de modo que al mirar cualquier fila se puede ver el estado de las estadisticas hasta ese punto de la simulacion.

11.1 contador_llegaron

Se incrementa en 1 al inicio de cada llamada a procesar_llegada_emp, antes de decidir si el empleado es atendido, encolado o rechazado. Cuenta **todos** los empleados que llegaron al sistema, sin excepcion.

11.2 contador_rt

Se incrementa solo en el tercer camino de procesar_llegada_emp, cuando la cola esta llena. Cuenta solo los rechazados. El porcentaje se calcula en la GUI: $n_{rt} / n_{lleg} * 100$.

11.3 contador_atendidos

Se incrementa en procesar_fin_atencion cada vez que un empleado termina su atencion. No cuenta a los que estan siendo atendidos ahora, solo a los que ya terminaron.

11.4 acum_espera y contador_espera

Son los mas delicados. La espera de un empleado se contabiliza en dos situaciones diferentes, pero siempre en el momento exacto en que el empleado **pasa de esperar a ser atendido**:

- **Atencion inmediata** (en procesar_llegada_emp): el empleado llego y habia terminal disponible. Su espera es 0. Se suma 0 a acum_espera y se incrementa contador_espera en 1.
- **Desde la cola** (en atenderCola_con_terminal): el empleado esperaba en cola. Su espera es reloj - hora_inicio_espera. Se suma esa diferencia a acum_espera y se incrementa contador_espera en 1.

El promedio de espera en cualquier fila es: $\text{acum_espera} / \text{contador_espera}$. Este promedio incluye a todos los empleados del sistema, incluso los que esperaron 0 minutos, por lo que representa el promedio real del sistema.

Los rechazados (RT) nunca entran en acum_espera ni en contador_espera: se fueron sin esperar y sin ser atendidos. Solo afectan contador_rt y contador_llegaron.

12. Que devuelve la simulacion y en que formato

Al terminar el loop, **simular()** devuelve **vector_estado**: una lista de diccionarios donde cada elemento representa una fila de la tabla.

Cada diccionario contiene:

- Datos de identificacion: num, reloj, evento.
- Proximos eventos: prox_emp, prox_tec.
- Estado de terminales: listas de 4 elementos (term_est, term_pend, term_fin).
- Estado del tecnico: fin_mant, tec_est, tec_term.
- Longitud de la cola: cola.
- Contadores acumulados: n_at, n_lleg, n_rt, n_esp, acum_esp.
- Numeros aleatorios generados en este evento: rnd_le, t_le, rnd_at, t_at, etc.
- Snapshot de empleados: emp (dict con el estado de cada empleado activo).

Todos los valores son **crudos**: floats, enteros, booleanos o None. El formateo se hace en la GUI al momento de mostrar cada celda, nunca en la simulacion.

Ademas de vector_estado, quedan disponibles como variables del modulo: **sim.reloj** (tiempo del ultimo evento), **sim.iteracion** (eventos procesados), y todos los contadores finales que la GUI usa para las tarjetas de estadisticas.

13. Flujo completo encadenado

Para cerrar, el recorrido completo de la simulacion de principio a fin:

- 1. resetear_estado()** — se limpian todas las variables. Las 4 terminales quedan Libres con pendiente=True. El tecnico queda Descansando.
- 2. simular(tiempo_max)** — se generan los dos primeros eventos: la primera llegada de empleado y la primera llegada del tecnico. Se guarda la fila de Inicializacion. El loop arranca.
- 3. El loop extrae el evento con menor tiempo del heap.** Generalmente la primera llegada de empleado ocurre antes que la del tecnico (media 2 min vs media 60 min).

4. procesar_llegada_emp() — el empleado llega. Si hay terminal libre, va directo. Se genera la proxima llegada de empleado y se inserta en el heap. Se guarda la fila.

5. El loop sigue extrayendo eventos. Llegan mas empleados, algunos van directo a terminales, otros esperan. El heap va creciendo y decreciendo segun se programan y procesan eventos.

6. En algun momento ocurre un fin_atencion. La terminal se libera. atenderColaConTerminal decide si el tecnico (si esta esperando) o el primer empleado de la cola toma esa terminal. Se guarda la fila.

7. Despues de ~60 minutos, llega el tecnico. procesar_llegada_tec lo manda a la primera terminal libre con pendiente. Si no hay libre, queda esperando. Se guarda la fila.

8. El tecnico va terminando terminales. Cada fin_manten busca la siguiente. Cuando termina la ronda completa, se resetean los pendientes y se programa la proxima llegada del tecnico.

9. El loop corta cuando el reloj supera tiempo_max o se alcanzan 99.999 iteraciones.

10. simular() devuelve vector_estado con todas las filas generadas. La GUI lee ese resultado y lo muestra en la tabla.

14. Arquitectura de funciones — quien llama a quien

Cada funcion del archivo tiene un rol bien definido. Entender esos roles y como se relacionan entre si es clave para poder explicar el codigo con claridad.

14.1 Loop principal

simular() es el unico punto de entrada. Inicializa el sistema, corre el loop y en cada iteracion llama al procesador que corresponde segun el tipo de evento extraido del heap. Nadie llama a simular() excepto main.py.

14.2 Procesadores de eventos

Son llamados **unicamente por el loop principal**. Cada uno maneja un tipo de evento, modifica el estado global y delega trabajo a las funciones de asignacion y consulta.

— **procesar_llegada_emp()**: decide si el empleado va a una terminal (llama a asignar_empleado_a_terminal), espera en cola, o se va.

— **procesar_fin_atencion(terminal_id)**: libera la terminal y llama a atenderColaConTerminal para resolver que sigue.

— **procesar_llegada_tec()**: decide si el tecnico va a una terminal (llama a asignar_tecnico_a_terminal), espera, o se reprograma.

— **procesar_fin_manten(terminal_id)**: libera la terminal, decide si el tecnico va a la siguiente terminal, espera, o termina la ronda.

14.3 Funciones de asignacion

Son llamadas por los procesadores y por `atenderColaConTerminal`. Modifican el estado del sistema y programan eventos en el heap.

— **asignarEmpleadoATerminal(`emp_id`, `terminal`)**: marca la terminal como Ocupada, cambia el estado del empleado a SA, genera el tiempo de atencion con `gen_atencion()` y programa `fin_atencion` en el heap.

— **asignarTecnicoATerminal(`terminal`)**: marca la terminal como Siendo mantenida, cambia el estado del tecnico a Realizando Mantenimiento, genera el tiempo de mantenimiento con `gen_mantenimiento()` y programa `fin_manten` en el heap.

— **atenderColaConTerminal(`terminal`)**: decide con prioridad que pasa cuando una terminal queda libre. Tecnico primero si corresponde, empleado de la cola segundo, nadie tercero. Llama a las dos funciones anteriores segun el caso.

14.4 Funciones de consulta

Solo **leen** el estado global, nunca lo modifican. Son llamadas por los procesadores y por `atenderColaConTerminal` para tomar decisiones.

— **terminalLibreParaEmp()**: busca terminal libre con round-robin (para empleados).

— **terminalLibreConPendiente()**: busca terminal libre con `pendiente=True` (para el tecnico).

— **hayPendienteOcupada()**: pregunta si hay alguna terminal ocupada con pendiente.

— **empEnCola()**: devuelve cuantos empleados hay esperando en la cola.

14.5 Generadores de variables aleatorias

Son llamados por las funciones de asignacion y directamente por los procesadores cuando necesitan programar una llegada futura. Cada uno devuelve (`rnd`, tiempo).

— **genLlegadaEmp()**: exponencial negativa para tiempo entre llegadas de empleados.

— **gen_atencion()**: uniforme para tiempo de atencion en terminal.

— **gen_mantenimiento()**: uniforme para tiempo de mantenimiento por terminal.

— **genLlegadaTec()**: uniforme para tiempo entre llegadas del tecnico.

— **trunc2(`x`)**: trunca a 2 decimales, usada por todos los generadores anteriores.

14.6 Registro de estado

Llamadas al final de cada procesador, una vez que el estado ya fue modificado.

— **snapshotEmpleados()**: foto del estado de todos los empleados activos en este momento exacto. Devuelve un dict `{col: (estado, hora_inicio_espera, terminal_id)}`.

— **guardarFila(evento, `rnds`)**: guarda una fila completa en `vector_estado` con todos los valores del estado actual y los RNDs generados en ese evento.

14.7 Cola de eventos (heap)

Usadas por procesadores y funciones de asignacion para insertar eventos, y por el loop para extraerlos.

- **push_evento(tiempo, tipo, eid)**: inserta un evento en el heap.
- **siguiente_evento()**: extrae el evento con menor tiempo del heap.
- **tiempo_de(tipo)**: consulta el proximo tiempo de un tipo de evento sin extraerlo. Se usa para mostrar las columnas Prox. Evento en la tabla.

14.8 Reset

- **resetear_estado()**: llamada por main.py antes de cada simulacion. Reinicia todas las variables globales a su estado inicial para que la nueva simulacion arranque desde cero sin residuos de la anterior.

Regla general del diseno: los procesadores son los unicos que saben que tipo de evento ocurrio. Las funciones de asignacion y consulta no saben desde que procesador las llaman — solo hacen su tarea y devuelven el resultado.